

**DESIGN AND DEVELOP A GPS-BASED VEHICLE TRACKING AND
FLEET MANAGEMENT SYSTEM. THE SYSTEM ENABLES REAL-
TIME VEHICLE TRACKING AND EFFICIENT FLEET MONITORING
FOR TRANSPORTATION MANAGEMENT.**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B – 24MER030

MOHAMMED SABEER S – 24MER031

PRAAVIN RAJAA G – 24MER039

PRASANNA T – 24MER042

DEVA PRIYAN R – 24MEL064

SATHISH KUMAR K – 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The GPS-Based Vehicle Tracking and Fleet Management System is designed to provide real-time monitoring and efficient management of vehicles used in transportation and logistics. The system uses a GPS module to continuously determine the vehicle's location, speed, and movement. This information is processed by a microcontroller and transmitted to a remote monitoring platform through a GSM or Internet communication module.

The fleet operator can view the vehicle's current location and track its movement on a monitoring dashboard. The system can also provide alerts for unauthorized movement, overspeeding, route deviation, and emergency situations. By providing accurate and continuous vehicle information, the system helps improve fleet utilization, reduce fuel consumption, increase vehicle security, and optimize transportation operations.

Overall, the proposed system provides a cost-effective, reliable, and automated solution for real-time vehicle tracking and fleet management.

2. Problem Statement

Traditional vehicle and fleet management systems often depend on manual monitoring, which makes it difficult to continuously track vehicles and manage transportation operations efficiently. Fleet operators may not have accurate information about the vehicle's current location, speed, route, or unexpected stops.

Lack of real-time tracking can lead to route deviations, fuel wastage, delays, unauthorized vehicle usage, and difficulty in responding to emergencies. Manual monitoring also increases the workload of fleet managers and reduces overall operational efficiency.

Therefore, there is a need for an automated GPS-based vehicle tracking and fleet management system that can continuously monitor vehicle location and movement, transmit real-time data, and provide alerts to fleet operators for effective transportation management.

3. Proposed Solution

The proposed system is a GPS-Based Vehicle Tracking and Fleet Management System that provides real-time monitoring of vehicles. A GPS module installed in the vehicle continuously determines its location, speed, and movement information.

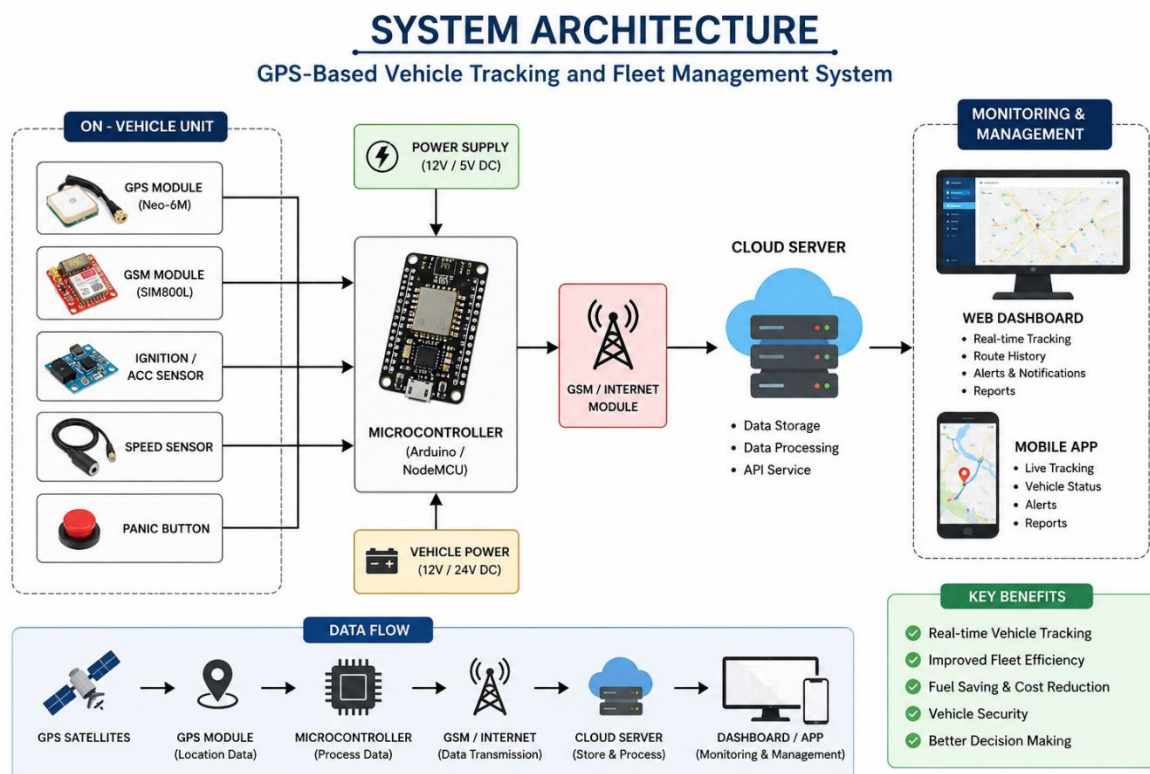
A microcontroller collects and processes the GPS data, while a GSM/Wi-Fi communication module transmits the information to a cloud server or monitoring application. The fleet manager can view the vehicle's current location and travel route through a dashboard.

The system can also generate alerts for overspeeding, route deviation, unauthorized movement, and emergency conditions. This helps fleet operators improve vehicle security, reduce fuel consumption, optimize routes, minimize delays, and manage multiple vehicles efficiently.

Main Functions

- Real-time vehicle location tracking
- GPS-based route monitoring
- Speed monitoring and overspeed alerts
- Route deviation detection
- Remote fleet monitoring
- Emergency and unauthorized movement alerts
- Centralized vehicle data management
- Improved transportation efficiency

4. System Architecture

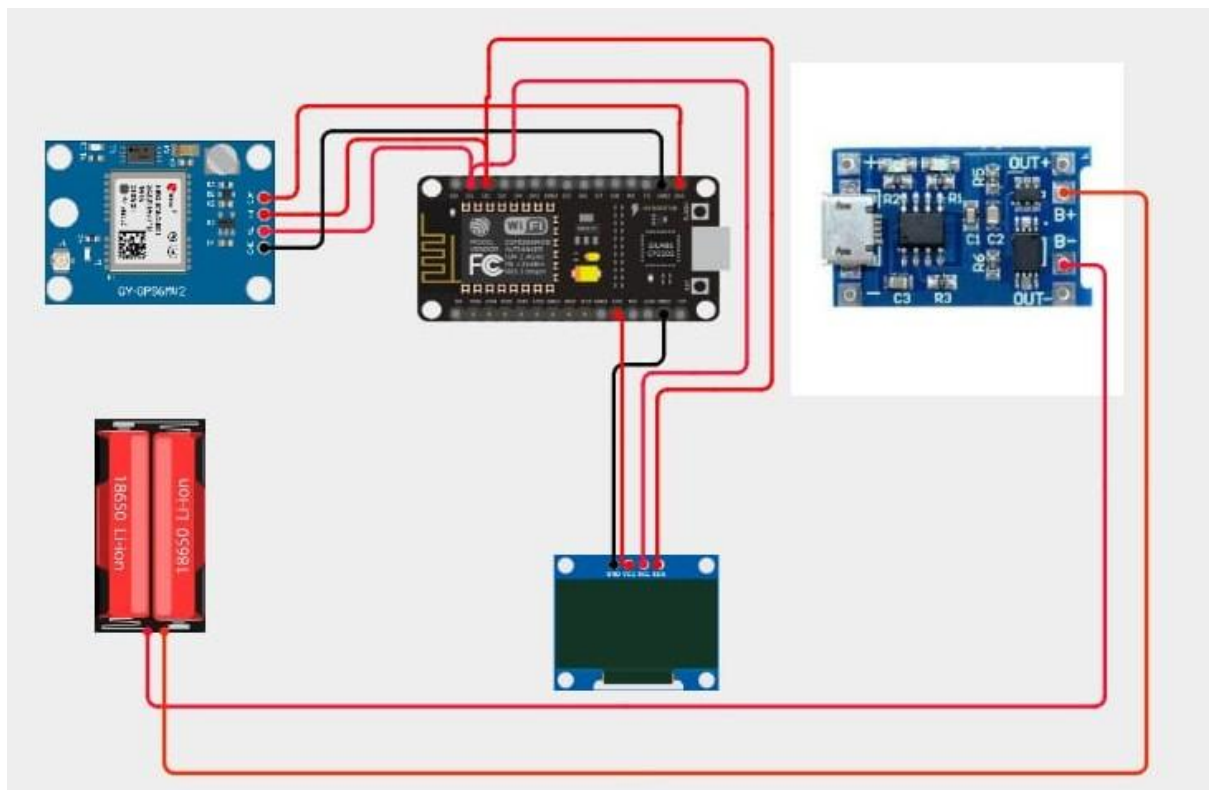


5. Circuit Table

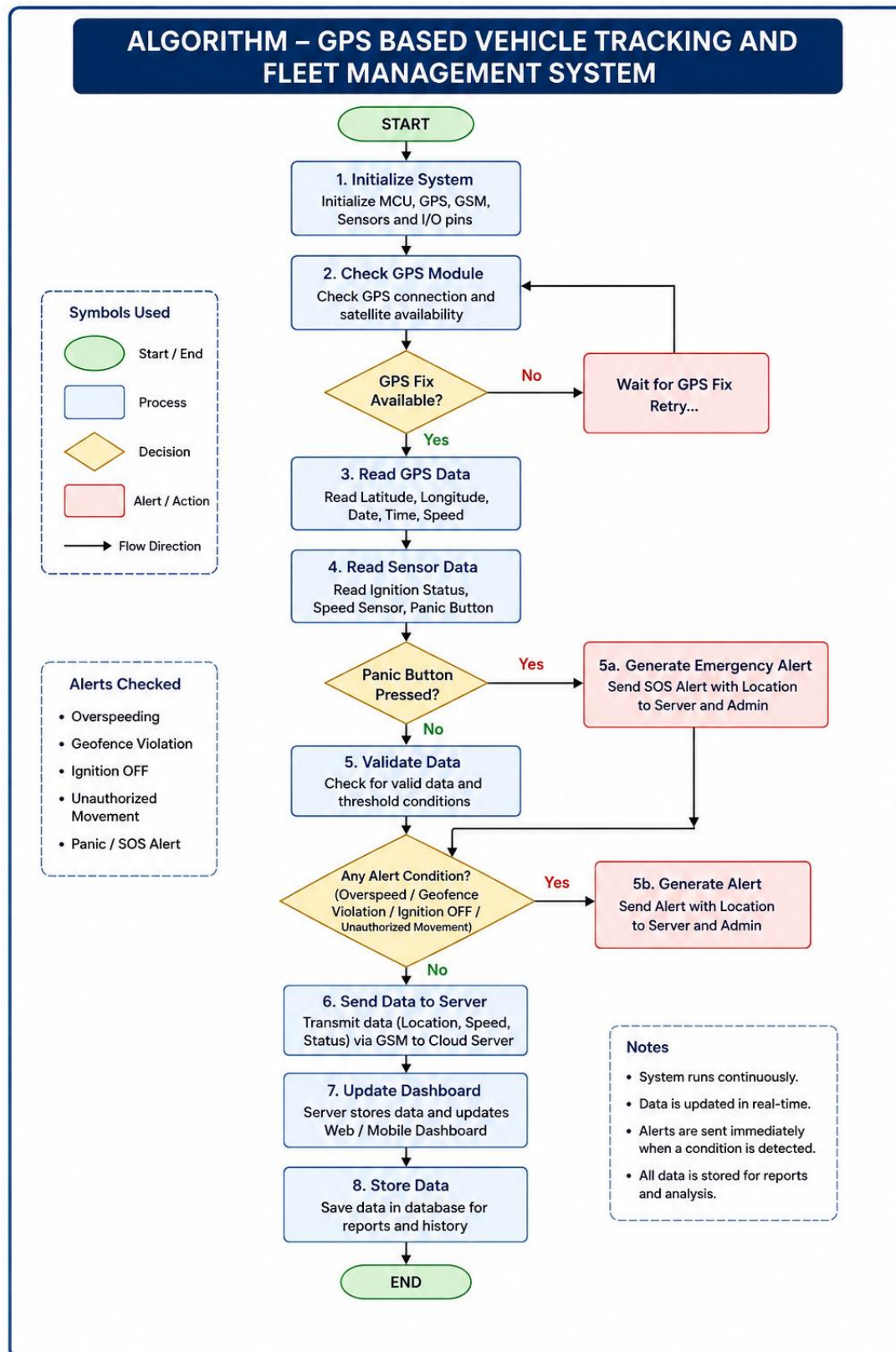
S.No.	Component	Pin/Terminal	Connection	Purpose
1	GPS Module (NEO-6M)	VCC	5V	Power supply
		GND	GND	Ground
		TX	MCU RX	Sends GPS location data
		RX	MCU TX	Receives GPS commands
2	GSM Module (SIM800L)	VCC	External supply	GSM power
		GND	GND	Ground
		TX	MCU RX	Sends communication data
		RX	MCU TX	Receives commands
3	Microcontroller (Arduino/NodeMCU)	VIN/5V	Power supply	Controller power
		GND	GND	Common ground
4	Ignition Sensor	OUT	Digital GPIO	Detects vehicle ON/OFF
5	Speed Sensor	OUT	Digital GPIO	Measures vehicle speed
6	Panic Button	One terminal	Digital GPIO	Emergency alert
		Other terminal	GND	Button return
7	Buzzer/LED	Positive	Digital GPIO	Alert indication
		Negative	GND	Ground
8	Power Supply	+V	MCU & modules	Provides required DC voltage

S.No.	Component	Pin/Terminal	Connection	Purpose
		GND	Common GND	Completes circuit

6. Circuit Design



7. Algorithm



8. Coding

```
// =====  
  
// GPS VEHICLE TRACKING & FLEET MONITORING SYSTEM  
  
// ESP8266 + NEO-6M + ADAFRUIT IO  
  
//  
  
// Features:  
  
// 1. Latitude  
  
// 2. Longitude  
  
// 3. Speed  
  
// 4. Number of satellites  
  
// 5. Altitude  
  
// 6. GPS Map location  
  
// 7. Adafruit IO upload every 30 seconds  
  
// =====  
  
#include <ESP8266WiFi.h>  
#include "AdafruitIO_WiFi.h"  
#include <TinyGPSPlus.h>  
#include <SoftwareSerial.h>  
  
// =====  
  
// WIFI SETTINGS  
  
// =====  
  
#define WIFI_SSID "HITMAN"  
#define WIFI_PASS "123456789"  
  
// =====  
  
// ADAFRUIT IO SETTINGS
```

```
// =====
```

```
#define IO_USERNAME "sathishkumar_67"
```

```
#define IO_KEY      "aio_zGJB61OPmoc4vVW2SSfURReqbBFu"
```

```
// =====
```

```
// ADAFRUIT IO CONNECTION
```

```
// =====
```

```
AdafruitIO_WiFi io(
```

```
  IO_USERNAME,
```

```
  IO_KEY,
```

```
  WIFI_SSID,
```

```
  WIFI_PASS
```

```
);
```

```
// =====
```

```
// ADAFRUIT IO FEEDS
```

```
// =====
```

```
AdafruitIO_Feed *latitudeFeed =
```

```
  io.feed("latitude");
```

```
AdafruitIO_Feed *longitudeFeed =
```

```
  io.feed("longitude");
```

```
AdafruitIO_Feed *speedFeed =
```

```
  io.feed("speed");
```

```
AdafruitIO_Feed *satelliteFeed =
```

```
  io.feed("satellites");
```

```
AdafruitIO_Feed *locationFeed =
```

```
  io.feed("gps-location");
```

```
// =====
```

```
// GPS PINS
```

```
// =====
```

```
//
```

```
// NEO-6M TX -> ESP8266 D5 / GPIO14
```

```
// NEO-6M RX -> ESP8266 D6 / GPIO12
```

```
// NEO-6M VCC -> 3.3V
```

```
// NEO-6M GND -> GND
```

```
//
```

```
// SoftwareSerial(RX, TX)
```

```
//
```

```
#define GPS_RX_PIN 14
```

```
#define GPS_TX_PIN 12
```

```
SoftwareSerial gpsSerial(
```

```
  GPS_RX_PIN,
```

```
  GPS_TX_PIN
```

```
);
```

```
// =====
```

```
// GPS OBJECT
```

```
// =====
```

```
TinyGPSPlus gps;
```

```
// =====
```

```
// UPLOAD TIMER
```

```
// =====
```

```
unsigned long lastUpload = 0;
```

```
// 30 seconds
```

```
const unsigned long uploadInterval = 30000;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  // -----
```

```
  // SERIAL MONITOR
```

```
  // -----
```

```
  Serial.begin(115200);
```

```
  delay(1000);
```

```
  Serial.println();
```

```
  Serial.println("=====");
```

```
  Serial.println("  GPS VEHICLE TRACKING SYSTEM");
```

```
Serial.println("=====");
Serial.println("ESP8266 + NEO-6M + Adafruit IO");
Serial.println();

// -----
// GPS START
// -----

gpsSerial.begin(9600);

Serial.println("GPS started");
Serial.println("GPS Baud Rate : 9600");
Serial.println();

Serial.println("GPS CONNECTION");
Serial.println("NEO-6M TX -> D5 / GPIO14");
Serial.println("NEO-6M RX -> D6 / GPIO12");
Serial.println("NEO-6M VCC -> 3.3V");
Serial.println("NEO-6M GND -> GND");
Serial.println();

// -----
// ADAFRUIT IO
// -----

Serial.println("Connecting to Adafruit IO...");

io.connect();
```

```

while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}

Serial.println();
Serial.println("Adafruit IO connected!");
Serial.println();

Serial.println("=====");
Serial.println("SYSTEM READY");
Serial.println("=====");
Serial.println("GPS data is being read continuously.");
Serial.println("Adafruit IO upload interval: 30 seconds");
Serial.println();
}

// =====
// LOOP
// =====

void loop()
{
    // -----
    // KEEP ADAFRUIT IO CONNECTION ALIVE
    // -----

    io.run();

```

```

// -----
// READ GPS DATA CONTINUOUSLY
// -----

while (gpsSerial.available() > 0)
{
    char c = gpsSerial.read();

    gps.encode(c);
}

// -----
// DISPLAY GPS DATA WHEN LOCATION UPDATES
// -----

if (gps.location.isUpdated())
{
    Serial.println();
    Serial.println("-----");
    Serial.println("CURRENT GPS DATA");
    Serial.println("-----");

    // =====
    // LATITUDE
    // =====

    Serial.print("Latitude    : ");

```

```
if (gps.location.isValid())
```

```
{
```

```
    Serial.println(
```

```
        gps.location.lat(),
```

```
        6
```

```
    );
```

```
}
```

```
else
```

```
{
```

```
    Serial.println("INVALID");
```

```
}
```

```
// =====
```

```
// LONGITUDE
```

```
// =====
```

```
Serial.print("Longitude   : ");
```

```
if (gps.location.isValid())
```

```
{
```

```
    Serial.println(
```

```
        gps.location.lng(),
```

```
        6
```

```
    );
```

```
}
```

```
else
```

```
{
```

```
    Serial.println("INVALID");
```

```
}
```

```
// =====  
// SPEED  
// =====
```

```
Serial.print("Speed      : ");
```

```
if (gps.speed.isValid())
```

```
{
```

```
    Serial.print(
```

```
        gps.speed.kmph(),
```

```
        2
```

```
    );
```

```
    Serial.println(" km/h");
```

```
}
```

```
else
```

```
{
```

```
    Serial.println("INVALID");
```

```
}
```

```
// =====  
// SATELLITES  
// =====
```

```
Serial.print("Satellites  : ");
```

```
if (gps.satellites.isValid())
```

```
{
```

```
        Serial.println(
            gps.satellites.value()
        );
    }
    else
    {
        Serial.println("INVALID");
    }

    // =====
    // ALTITUDE
    // =====

    Serial.print("Altitude    : ");

    if (gps.altitude.isValid())
    {
        Serial.print(
            gps.altitude.meters(),
            2
        );

        Serial.println(" m");
    }
    else
    {
        Serial.println("INVALID");
    }
}
```

```
Serial.println("-----");
```

```
// =====
```

```
// CHECK 30 SECOND UPLOAD TIMER
```

```
// =====
```

```
if (millis() - lastUpload >= uploadInterval)
```

```
{
```

```
    lastUpload = millis();
```

```
// =====
```

```
// CHECK GPS LOCATION
```

```
// =====
```

```
if (gps.location.isValid())
```

```
{
```

```
    // -----
```

```
    // GET GPS VALUES
```

```
    // -----
```

```
double latitude =
```

```
    gps.location.lat();
```

```
double longitude =
```

```
    gps.location.lng();
```

```
double speed = 0;
```

```
int satellites = 0;
```

```
double altitude = 0;
```

```
// -----
```

```
// SPEED
```

```
// -----
```

```
if (gps.speed.isValid())
```

```
{
```

```
    speed = gps.speed.kmph();
```

```
}
```

```
// -----
```

```
// SATELLITES
```

```
// -----
```

```
if (gps.satellites.isValid())
```

```
{
```

```
    satellites =
```

```
        gps.satellites.value();
```

```
}
```

```
// -----
```

```
// ALTITUDE
```

```
// -----
```

```
if (gps.altitude.isValid())
```

```
{
```

```
    altitude =
```

```

        gps.altitude.meters();
    }

// =====

// START UPLOAD

// =====

Serial.println();
Serial.println("*****");
Serial.println("UPLOADING DATA TO ADAFRUIT IO");
Serial.println("*****");

// =====

// LATITUDE FEED

// =====

latitudeFeed->save(latitude);

Serial.print("Latitude uploaded : ");
Serial.println(
    latitude,
    6
);

// =====

// LONGITUDE FEED

// =====

longitudeFeed->save(longitude);

```

```
Serial.print("Longitude uploaded : ");
```

```
Serial.println(
```

```
    longitude,
```

```
    6
```

```
);
```

```
// =====
```

```
// SPEED FEED
```

```
// =====
```

```
speedFeed->save(speed);
```

```
Serial.print("Speed uploaded   : ");
```

```
Serial.print(
```

```
    speed,
```

```
    2
```

```
);
```

```
Serial.println(" km/h");
```

```
// =====
```

```
// SATELLITE FEED
```

```
// =====
```

```
satelliteFeed->save(satellites);
```

```
Serial.print("Satellites uploaded: ");
```

```
Serial.println(satellites);
```

```
// =====  
// GPS MAP LOCATION  
// =====
```

```
locationFeed->save(  
    speed,  
    latitude,  
    longitude,  
    altitude  
);
```

```
Serial.println(  
    "GPS location uploaded to map"  
);
```

```
// =====  
// UPLOAD COMPLETE  
// =====
```

```
Serial.println("-----");  
Serial.println("UPLOAD COMPLETE");  
Serial.println("Next upload in 30 seconds");  
Serial.println("-----");  
Serial.println();  
}  
else  
{  
    // =====
```

```
// GPS NOT VALID

// =====

Serial.println();
Serial.println("*****");
Serial.println("GPS LOCATION INVALID");
Serial.println("Data NOT uploaded");
Serial.println("Waiting for GPS fix...");
Serial.println("*****");
Serial.println();
}
}
}

// -----

// SMALL DELAY

// -----

delay(10);
}
```

9. System Working

1. System Initialization: The microcontroller, GPS module, GSM/Internet module, and sensors are powered ON and initialized.
2. GPS Location Detection: The GPS module receives signals from satellites and determines the vehicle's latitude, longitude, speed, and time.
3. Data Processing: The microcontroller collects the GPS and vehicle sensor data and processes it for transmission.
4. Data Transmission: The processed vehicle information is transmitted to the cloud server through the GSM/Internet communication module.
5. Cloud Data Storage: The server receives and stores the vehicle information for real-time monitoring and future analysis.
6. Dashboard Monitoring: The fleet operator can view the vehicle's live location, speed, route, and status through a web or mobile dashboard.
7. Alert Generation: The system checks for conditions such as overspeeding, route deviation, unauthorized movement, or emergency button activation and sends alerts to the fleet manager.
8. Continuous Tracking: The system continuously repeats the tracking and data transmission process, allowing the fleet operator to monitor vehicles in real time.

10. Bill of Materials

S.No.	Component	Specification	Quantity	Purpose
1	Microcontroller	Arduino Uno / ESP32	1	Controls and processes system data
2	GPS Module	NEO-6M GPS	1	Provides vehicle location and speed
3	GSM Module	SIM800L / SIM900	1	Sends tracking data to the server
4	Vehicle Speed Sensor	Hall-effect / wheel sensor	1	Measures vehicle speed
5	Ignition Sensor	Digital input sensor	1	Detects vehicle ON/OFF status

S.No.	Component	Specification	Quantity	Purpose
6	Panic Button	Push-button switch	1	Sends emergency alert
7	Buzzer	5V Buzzer	1	Provides warning indication
8	LED Indicators	Red/Green LEDs	2–3	Indicates system status
9	Voltage Regulator	12V to 5V DC-DC converter	1	Provides regulated power
10	Power Supply	Vehicle 12V DC	1	Main power source
11	GSM Antenna	Compatible antenna	1	Improves GSM communication
12	GPS Antenna	GPS antenna	1	Improves satellite reception
13	Connecting Wires	Jumper/automotive wires	As required	Electrical connections
14	PCB/Breadboard	Prototype board	1	Circuit assembly
15	Cloud/Database Platform	IoT cloud/server	1	Stores tracking data
16	Monitoring Device	PC/Smartphone	1	Displays vehicle location and status
17	Enclosure	Protective project box	1	Protects electronic components

11. Prototype Implementation

The prototype of the GPS-Based Vehicle Tracking and Fleet Management System was developed by integrating a microcontroller, GPS module, GSM/Internet communication module, and vehicle sensors. The GPS module was connected to the microcontroller to obtain the vehicle's latitude, longitude, speed, and time information.

The microcontroller processes the collected data and sends it to a cloud server through the GSM/Internet module. A web or mobile dashboard is used to display the vehicle's real-time location, speed, route, and operating status.

For demonstration, the system was mounted on a small vehicle model. A speed sensor and ignition input were added to monitor vehicle operation, while a panic button was included for emergency alerts. The system was tested under different movement and operating conditions to verify location tracking, data transmission, and alert generation.

Prototype Steps

1. Assemble the microcontroller, GPS, GSM/Internet module, and sensors.
2. Connect the modules according to the circuit design.
3. Program the microcontroller to collect and process vehicle data.
4. Configure the communication module for cloud data transmission.
5. Develop the monitoring dashboard for live vehicle tracking.
6. Test GPS location, speed monitoring, and vehicle status.
7. Test alerts for overspeeding, route deviation, and emergency conditions.
8. Continuously monitor the vehicle and record tracking data for analysis.



12. Results & Performance Analysis

Results

The developed GPS-Based Vehicle Tracking and Fleet Management System successfully tracked the vehicle's location and transmitted the data to the monitoring dashboard. The system displayed important information such as latitude, longitude, speed, route, and vehicle status in real time. Alert functions were also tested for conditions such as overspeeding, route deviation, and emergency button activation.

Performance Analysis

The prototype provided reliable vehicle tracking and continuous data transmission during testing. GPS information was obtained accurately when sufficient satellite signals were available, while the GSM/Internet module enabled remote monitoring. The system reduced the need for manual vehicle tracking and improved fleet monitoring efficiency.

Overall Performance

The prototype demonstrates that the proposed system can provide effective real-time vehicle tracking, remote fleet monitoring, and timely alerts. It can help transportation operators improve vehicle security, route management, fleet utilization, and overall operational efficiency.

